

MCPs y Skills

Claude Code en Acción

Etapa de Nivelación · Devs Mid-Senior · 40 min

Agenda

01

Por qué MCPs cambian el juego

5 min

02

Arquitectura MCP: lo que importa

8 min

03

Setup real con Claude Code

7 min

04

Skills: automatización de flujos

7 min

05

Práctica 1: Conectar un MCP server

7 min

06

Práctica 2: Crear tu primer Skill

6 min

00

Instalando Claude Code

Agent

~/scratch \$ ~/.local/bin/claude --version

Version: 2.1.52

Location: ~/.local/bin/claude

Next: Run `claude --help` to get started

✓ Installation complete!

arkatechie@192 scratch % ~/.local/bin/claude --version

2.1.52 (Claude Code)

arkatechie@192 scratch %

Ask every time ^

Exit code 0

Everything looks great! 🎉

Next Steps

1. Make sure `~/.local/bin` is in your PATH — add this to your `~/.zshrc` if you haven't already:

bash

`export PATH="$HOME/.local/bin:$PATH"`

Then reload your shell:

bash

`source ~/.zshrc`

2. Start Claude Code by navigating to any project directory and running:

bash

`claude`

3. First Launch — you'll be prompted to log in with your Anthropic/Claude.ai account to authenticate.

Would you like help with anything else, like adding it to your PATH or getting started with your first session?

Good 👍 Bad 🗑️

📄 📄 📄 📄

Ask anything, @ to mention, / for workflows

+ ^ Fast ^ Claude Sonnet 4.6 (Thinking)

claude

Code review C

~

claude

Welcome to Claude Code v2.1.52



Let's get started.

Choose the text style that looks best with your terminal

To change this later, run `/theme`

> 1. Dark mode ✓

2. Light mode

3. Dark mode (colorblind-friendly)

4. Light mode (colorblind-friendly)

5. Dark mode (ANSI colors only)

6. Light mode (ANSI colors only)

1 function greet() {

2 - console.log("Hello, World!");

2 + console.log("Hello, Claude!");

3 }

Syntax theme: Monokai Extended (ctrl+t to disable)

🔍 🖼️ 📄 📄

File explorer

± View changes

🔍 🔍

01

Por qué MCPs cambian el juego

El contexto que todo dev mid-senior necesita en 5 minutos

claude



Claude Code v2.1.52

Welcome back Arkatechie!



Sonnet 4.6 · Claude Pro · arkatechie@gmail.com's
Organization

/Users/arkatechie

Tips for getting started

Run `/init` to create a `CLAUDE.md` file with instructions for Claude
Note: You have launched claude in your home directory. For the be...

Recent activity

No recent activity

▸ /agents

└ Agents dialog dismissed

▸ claude mcp list

● Bash(claude mcp list)

└ Running...

Bash command

```
claude mcp list
```

List configured MCP servers

This command requires approval

Do you want to proceed?

- 1. Yes
- 2. Yes, and don't ask again for: claude mcp:*
- 3. No

Esc to cancel · Tab to amend · ctrl+e to explain

    File explorer  View changes  

Antes de MCP: dos caminos, ninguno bueno

Camino A — Function calling ad-hoc

- ✗ Escribes la integración por proyecto, a mano
- ✗ Cero reutilización real entre proyectos
- ✗ Schema diferente en cada LLM provider
- ✗ Mantienes el wrapper tú mismo para siempre

Camino B — Agentes con tool use custom

- ✗ Cada proveedor tenía su propio esquema
- ✗ Cada IDE tenía su propio formato
- ✗ Fragmentación total del ecosistema
- ✗ Portabilidad prácticamente nula

MCP: el estándar que lo resuelve

Model Context Protocol — protocolo JSON-RPC 2.0 que define cómo un host (Claude Code, Claude Desktop, Cursor...) se comunica con servidores de herramientas de forma estandarizada

La analogía correcta

MCP es a los agentes lo que LSP es a los editores de código

Un protocolo de transporte agnóstico que permite que cualquier herramienta se conecte a cualquier agente

Write once

Un server MCP funciona en Claude Code, Claude Desktop, Cursor, Zed o cualquier host compatible

Ecosistema

GitHub, Slack, Notion, Postgres, Filesystem y docenas más — listos para usar hoy

Tus sistemas

Tus APIs internas se vuelven ciudadanos de primera clase en el flujo del agente

02

Arquitectura MCP: lo que importa

Tres capas, tres primitivos, un modelo de seguridad

Modelo de tres capas

HOST (Claude Code / Claude Desktop / Cursor)

- Mantiene sesión con el modelo
- Gestiona múltiples MCP Clients
- Aplica permisos y scope de aprobación



MCP CLIENT (una instancia por server)

- Conexión 1:1 con un MCP Server · Ciclo: initialize → tools/list → tools/call · Traduce capabilities a tool definitions



MCP SERVER

- Expone Tools / Resources / Prompts · Local (stdio) o remoto (SSE / HTTP Streamable) · Stateful o stateless

Punto crítico

El modelo NO ejecuta las tools directamente.

El Host decide si aprobar la ejecución — humano en el loop, o aprobación automática según política configurada.

Security model

- Servers corren con permisos del proceso. Sin sandboxing automático.
- Principio de menor privilegio: define explícitamente qué puede hacer cada server.
- Para servers de terceros: revisa el código antes de ejecutar.

Los tres primitivos

Tools

Lo que más vas a usar.
Funciones que el modelo puede invocar.

```
{
  "name": "query_db",
  "description": "Query SQL
de solo lectura",
  "inputSchema": {
    "type": "object",
    "properties": {
      "query": { "type": "string"
    },
    "db": { "enum": ["prod",
      "staging"] }
    }
  }
}
```

Resources

Datos con URI que el modelo puede leer —
archivos, endpoints, registros.

```
file:///schema.sql

postgres://localhost/
mydb/tables/users

https://api.internal/
docs/openapi.yaml

git://repo/HEAD/src/
```

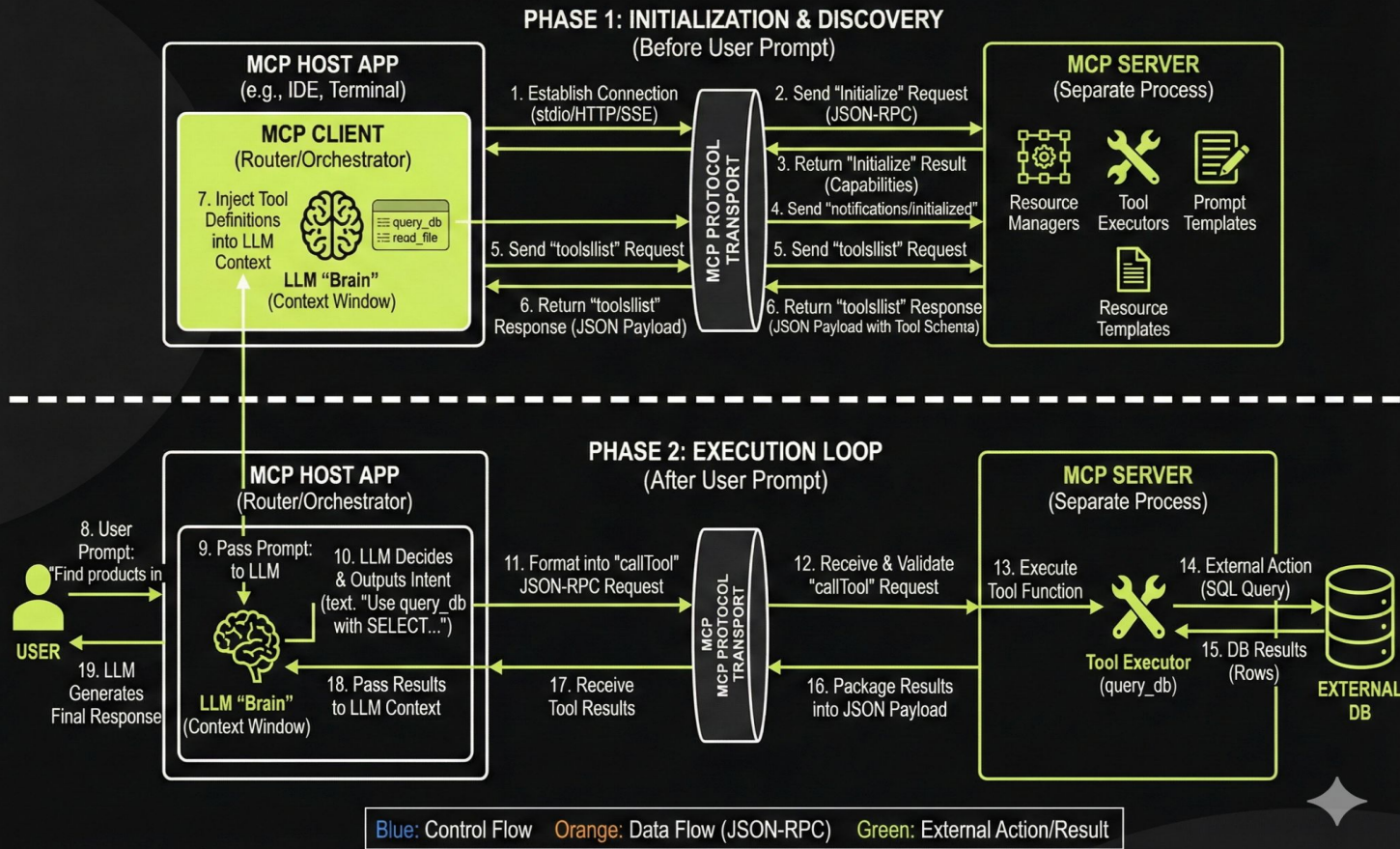
Prompts

Templates reutilizables con parámetros.
Estandarizan flujos repetitivos en el equipo.

Caso de uso típico:

- Generar PRD desde un template de equipo
- Code review con criterios propios
- Runbooks de incidentes parametrizados
- Onboarding con contexto de proyecto

MCP Server Communication



Transportes y ciclo de vida

Transporte	Descripción	Uso ideal
stdio	Procesos locales. El host levanta el server como subprocess. Default para CLI.	Producción local
SSE	Servers remotos, multi-tenant. Ideal para servicios compartidos.	Servicios remotos
HTTP Streamable	Nuevo en MCP 2025-11-05. Reemplaza SSE, más robusto para producción a escala.	Producción cloud ✓

Ciclo de vida de una tool call



↩ el resultado vuelve al modelo como tool result message — el ciclo puede reiniciarse

03

Setup real con Claude Code

Configuración, CLI y workflow dentro de una sesión

Hardcore AI by 30x: Kitchen analogy

INDIVIDUAL MCPs & APIs (The Kitchen Tools)



**GROCERY
DELIVERY API**
Service: Order
Ingredients



STOVE MCP
Tool: Heat & Cook



OVEN MCP
Tool: Bake



BLENDER MCP
Tool: Mix



KNIFE MCP
Tool: Chop

MAIN SKILL: PREPARING LASAGNA (The Workflow)

STEP 0: Plan & Order



AI AGENT
(The Chef)



RECIPE DATABASE API
Service: Fetch Instructions

STEP 1: Prep Sauce

Chop Veggies (Knife MCP)
Sauté & Simmer (Stove MCP)

STEP 2: Prepare Pasta Sheets

Boil Water & Pasta (Stove MCP)

STEP 3: Assemble

Layer ingredients (Agent Logic)

STEP 4: Bake

Bake in Oven (Oven MCP)

**SUB-SKILL:
MAKE FRESH PASTA**
(Mini-Workflow)

Mix Flour & Eggs
(Blender MCP)

Knead & Roll Dough
(Agent Logic)



FINAL PRODUCT

Jerarquía de config y CLI esencial

Dónde vive la configuración

`.claude/settings.json`

Proyecto — Commiteado. Lo ven todos.

`~/.claude.json`

Usuario — Global, solo tu máquina.

`~/.claude/settings.local.json`

Local — Secrets. Nunca al repo.

settings.json resultante

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y",
        "@modelcontextprotocol/server-filesystem",
        "/workspace"]
    }
  }
}
```

CLI esencial

```
# Agregar server (scope proyecto)
claude mcp add filesystem -- \
  npx -y @modelcontextprotocol/server-filesystem \
  /workspace

# Scope global (solo tu máquina)
claude mcp add --scope user github -- \
  npx -y @modelcontextprotocol/server-github \
  -e GITHUB_PERSONAL_ACCESS_TOKEN=$TOKEN

# Verificar estado
claude mcp list
claude mcp get github

# Dentro de una sesión
/mcp          # ver servers y status
/tools        # ver todas las tools disponibles
```


04

Skills: automatización de flujos de trabajo

Contexto comprimido + proceso versionado = comportamiento consistente

Qué es un Skill y cuándo crearlo

	Prompt ad-hoc	Skill
Dónde vive	En tu cabeza o un doc suelto	En el repo — versionado con git
Contexto	Debes recordarlo tú cada vez	Claude lo lee automáticamente
Consistencia	Varía entre sesiones	Comportamiento reproducible
Quién lo usa	Solo tú	Todo el equipo, de forma igual
Activación	Copy-paste manual	/skill nombre-del-skill

Crea un Skill cuando...

- ✓ Repites el flujo más de 3 veces
- ✓ El proceso tiene más de 4 pasos con contexto específico
- ✓ Quieres estandarizar cómo el equipo hace algo

No crees un Skill cuando...

- ✗ Es una tarea de una sola vez
- ✗ El proceso cambia tanto que quedaría desactualizado en días
- ✗ Un buen prompt en el chat es suficiente

Anatomía de un SKILL.md

Estructura de archivos

```
.claude/  
  skills/  
    mi-skill/  
      SKILL.md      ← instrucciones (obligatorio)  
      examples/     ← input/output de ejemplo  
      templates/    ← archivos base del skill
```

Cómo Claude Code ejecuta un Skill

- 1 Lee el SKILL.md completo antes de responder
- 2 Las instrucciones del skill tienen prioridad sobre el comportamiento default
- 3 Puede hacer preguntas si el skill lo indica — antes de ejecutar
- 4 Otro dev con el mismo repo y skill obtiene el mismo resultado

Secciones del SKILL.md

Cuándo usar este skill

Triggers claros. Qué pregunta o tarea lo activa.

Contexto necesario

Qué preguntar al usuario antes de ejecutar.

Proceso paso a paso

Instrucciones en orden. Claude las sigue secuencialmente.

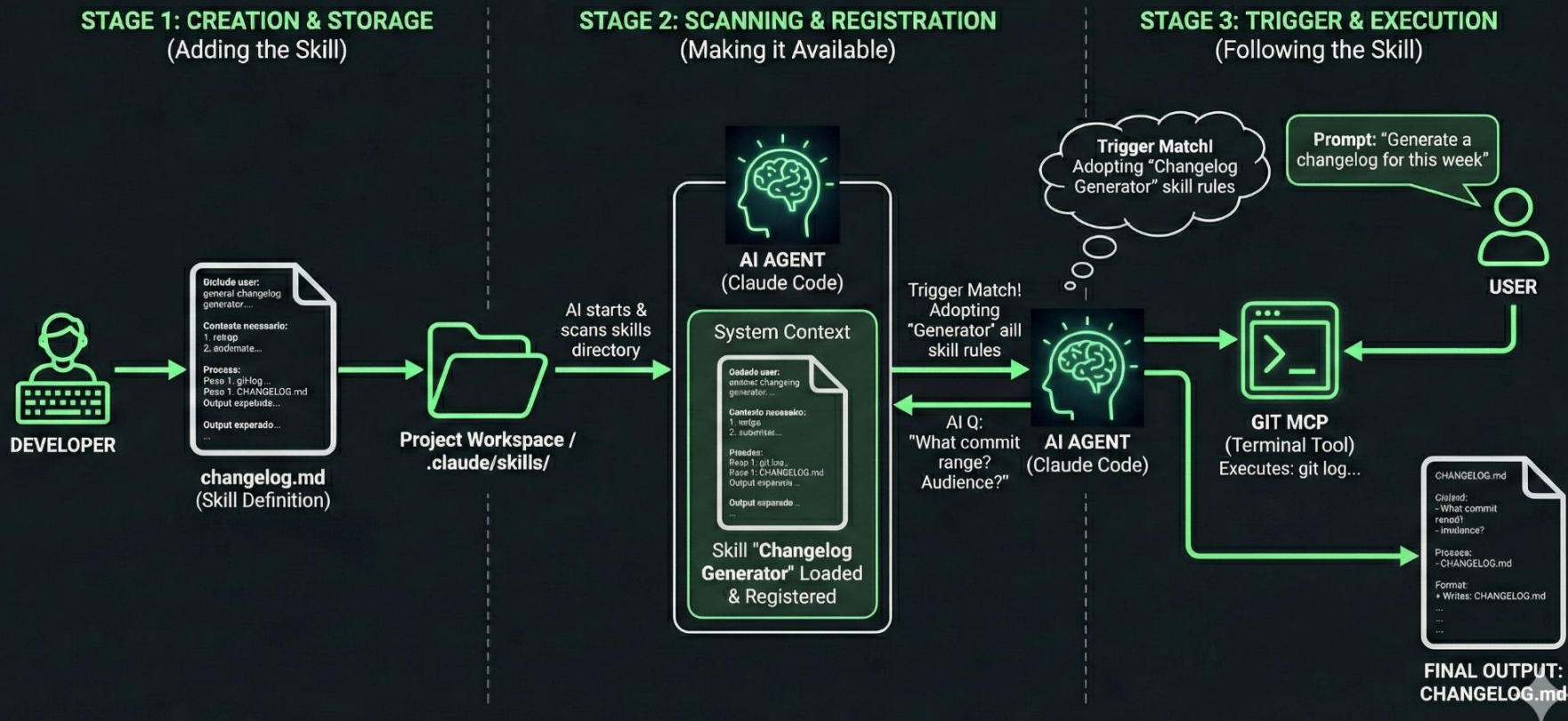
Output esperado

Formato exacto del resultado, con ejemplos si es posible.

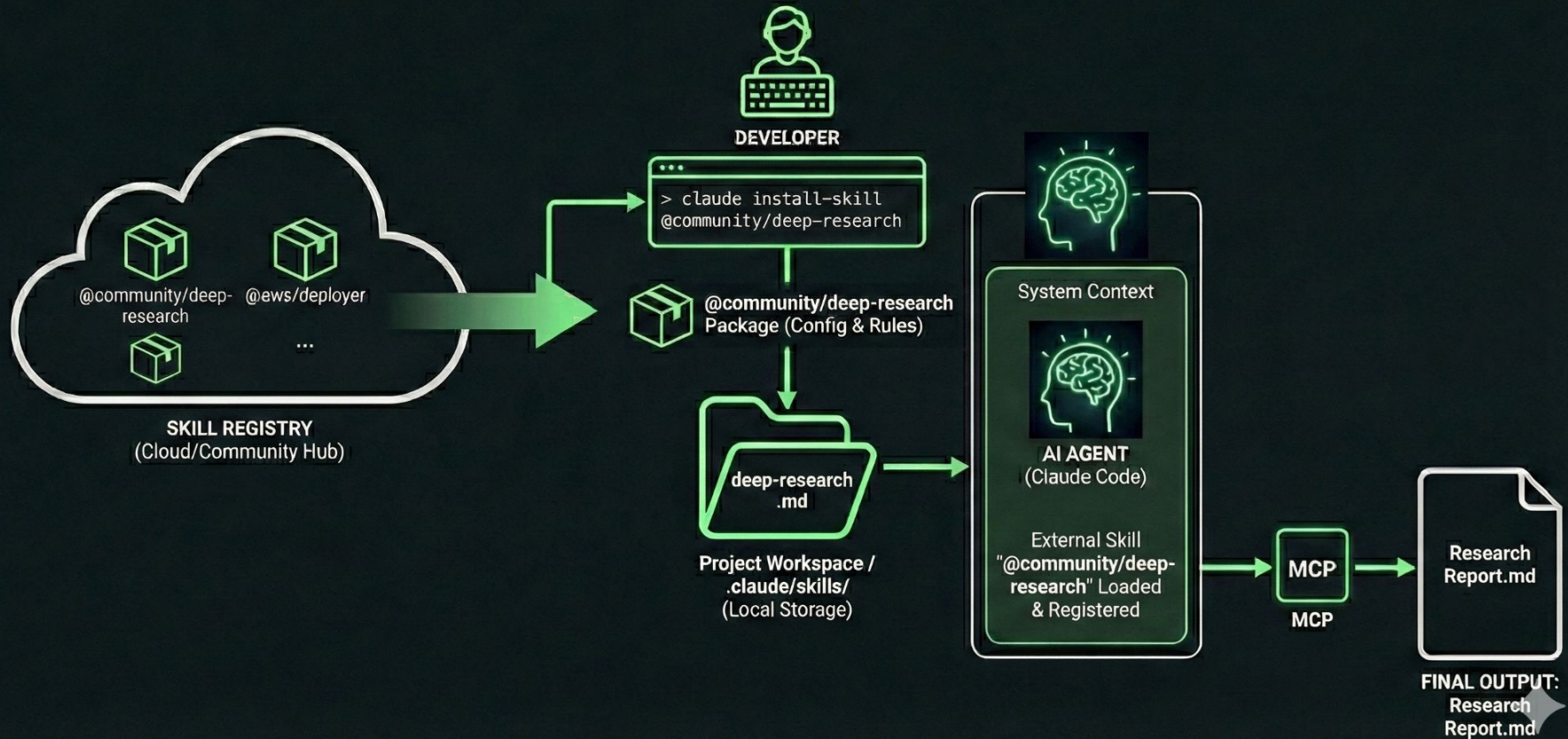
Errores comunes

Casos edge que Claude debe anticipar y manejar.

Hardcore AI by 30x: Skill Lifecycle - From File to Execution



Hardcore AI by 30x: Installing External Skills (The 'App Store' Pipeline)



05

Práctica 1: Conectar un MCP Server

MCP Filesystem → datos reales → reporte en Markdown

Setup y pasos

1 Instalar y agregar el server

```
mkdir mcp-practice && cd mcp-practice
mkdir -p .claude src data
claude mcp add filesystem -- \
  npx -y @modelcontextprotocol/server-fileSystem $(pwd)/data
claude mcp list # filesystem:✓ connected
```

2 Crear datos de prueba

```
cat > data/products.json << 'EOF'
[
  { "id": 1, "name": "Widget A", "stock": 150 },
  { "id": 2, "name": "Widget B", "stock": 3 },
  { "id": 3, "name": "Gadget X", "stock": 0 }
]
EOF
```

3 Sesión Claude Code — leer y reportar

```
claude
# Prompt:
# Lee data/products.json. ¿Cuáles tienen stock bajo (< 10)
# o agotado? Genera un reporte en markdown.
```

4 Escribir resultado al filesystem

```
# Prompt:
# Crea data/low-stock-report.md con el reporte.
# Incluye fecha, lista y recomendación de acción.
```



```
~/Work/tribu/claude-mcp-skills (0.692s)
```

```
claude mcp add filesystem -- npx -y @modelcontextprotocol/server-filesystem $(pwd)/data
```

Added stdio MCP server filesystem with command: npx -y @modelcontextprotocol/server-filesystem /Users/cristian.cordoba/Work/tribu/claude-mcp-skills/data to local config
File modified: /Users/cristian.cordoba/.claude.json [project: /Users/cristian.cordoba/Work/tribu/claude-mcp-skills]

```
~/Work/tribu/claude-mcp-skills (15.443s)
```

```
claude mcp list
```

Checking MCP server health...

```
filesystem: npx -y @modelcontextprotocol/server-filesystem /Users/cristian.cordoba/Work/tribu/claude-mcp-skills/data - ✓ Connected
```

```
~/Work/tribu/claude-mcp-skills (0.179s)
```

```
cat > data/products.json << 'EOF'
```

```
[
  { "id": 1, "name": "Widget A", "price": 29.99, "stock": 150 },
  { "id": 2, "name": "Widget B", "price": 49.99, "stock": 3 },
  { "id": 3, "name": "Gadget X", "price": 199.99, "stock": 0 },
  { "id": 4, "name": "Gadget Y", "price": 89.99, "stock": 45 }
]
EOF
```

```
"Lee data/products.json y crea data/low-stock-report.md con productos de stock bajo o agotado, fecha y recomendación."
```

• Read 1 file (ctrl+o to expand)

• Write(data/low-stock-report.md)

```
└ Wrote 30 lines to data/low-stock-report.md
  1 # Reporte de Stock Bajo y Agotado
  2
  3 **Fecha:** 2026-02-25
  4
  5 ---
  6
  7 ## Productos con Problemas de Stock
  8
  9 | ID | Producto | Precio | Stock | Estado |
 10 |---|-----|-----|-----|-----|
  ... +20 lines (ctrl+o to expand)
```

• Reporte creado en data/low-stock-report.md. Resumen de lo encontrado:

```
- Gadget X (ID: 3) - Stock: 0 -> Agotado
- Widget B (ID: 2) - Stock: 3 -> Stock bajo
```

Los otros dos productos (Widget A con 150 y Gadget Y con 45) están en niveles normales. El reporte incluye fecha, tabla de estado y recomendaciones de acción para cada producto problema.

```
/exit
└ See ya!
```


06

Práctica 2: Crear tu primer Skill

Generador de changelogs automático desde el historial git

Skill de Changelog — estructura y ejecución

Estructura del SKILL.md

Cuándo usar	Cuando pidan changelog, release notes o resumen de cambios
Contexto previo	Rango de commits · audiencia (técnica/usuarios) · formato salida
Paso 1	git log \$(git describe --tags)..HEAD --oneline --no-merges
Paso 2	Categorizar: Breaking / Features / Fixes / Perf / Internal
Paso 3	Generar según audiencia: técnico con SHA, usuarios con beneficios
Paso 4	Escribir CHANGELOG.md — agregar al inicio, conservar historial
Output	## [v1.x.x] - YYYY-MM-DD con emojis por categoría

Probarlo en 60 segundos

```
# 1. Crear la estructura
mkdir -p .claude/skills/changelog

# 2. Crear SKILL.md con el contenido del skill

# 3. Repo de prueba con commits reales
git init
git commit --allow-empty \
  -m "feat: add user authentication"
git commit --allow-empty \
  -m "fix: resolve token expiration bug"
git commit --allow-empty \
  -m "feat!: migrate to API v2 - breaking"
git commit --allow-empty \
  -m "perf: optimize database queries"

# 4. Activar el skill
claude
# dentro de la sesión:
```

Claude lee el SKILL.md → hace las preguntas → ejecuta git commands → genera el CHANGELOG.md con el formato acordado

Patrones de diseño que escalan

Skill como checklist con exit conditions

Define condiciones de parada explícitas. Claude se detiene y reporta si tests están en rojo, env vars no definidas, o hay TODOs sin resolver en código nuevo.

```
## Condiciones de salida
DETENTE si encuentras:
- [ ] Tests en rojo
- [ ] Env vars no definidas
- [ ] TODOs en código nuevo
```

Skill compuesto (Skills que invocan otros Skills)

Un Skill puede orquestar otros Skills en secuencia. Ideal para flujos completos como prepare-release: genera tests → actualiza docs → crea PR description.

```
## Proceso
1. Ejecuta /skill generate-tests
2. Ejecuta /skill update-docs
3. Genera PR description
```

Skill + MCP para workflows end-to-end

Combina Skills con MCP servers para workflows que cruzan sistemas: crear PR en GitHub, mover ticket en Jira y notificar en Slack — todo en una instrucción.

```
## MCP servers requeridos:
- github: crear PR
- jira: mover ticket
- slack: notificar canal
```

```
~/Work/tribu/claude-mcp-skills (0.157s)  
touch .claude/skills/changelog/SKILL.md
```

```
~/Work/tribu/claude-mcp-skills (3m 26.96s)  
nano .claude/skills/changelog/SKILL.md
```

```
~/Work/tribu/claude-mcp-skills (0.587s)  
git init
```

```
Initialized empty Git repository in /Users/cristian.cordoba/Work/tribu/claude-mcp-skills/.git/
```

```
~/Work/tribu/claude-mcp-skills git:(main) (0.437s)  
git commit --allow-empty -m "feat: add user authentication"
```

```
[main (root-commit) 2e8fbff] feat: add user authentication
```

/changelog

- User answered Claude's questions:
 - └ · ¿Para qué rango de commits quieres generar el changelog? → Últimos 20 commits
 - ¿A quién va dirigido el changelog? → Técnico/Interno
 - ¿En qué formato quieres la salida? → Markdown (Recommended)
- **Bash**(git log --oneline --no-merges -n 20 && echo "---" && git describe --tags --abbrev=0 2>/dev/null || echo "No tags found")
 - └ 7ee8d8c chore: update dependencies
 - fe70bf2 perf: optimize database queries
 - b50510b fix: resolve token expiration bug
 - ... +3 lines (ctrl+o to expand)
- **Bash**(git log --no-merges -n 20 --pretty=format:"%H|%s|%an|%ad" --date=short)
 - └ 7ee8d8ca4ec082736d1bb70425ced58dbd553abb|chore: update dependencies|cdcordobaa|2026-02-25
 - fe70bf22d5c1b0bd39a51e0c74d8a3fef9079ffa|perf: optimize database queries|cdcordobaa|2026-02-25
 - b50510beb49df237b3389579f6eb66c28014fab3|fix: resolve token expiration bug|cdcordobaa|2026-02-25
 - 2e8fbffe85f86979891242ddb1dec2f731d01c6e|feat: add user authentication|cdcordobaa|2026-02-25
- **Bash**(git log --no-merges -n 20 --pretty=format:"%H" | while read sha; do echo "=== \$sha ==="; git show --stat --no-patch "\$sha" | tail -5; done)
 - └ === 7ee8d8ca4ec082736d1bb70425ced58dbd553abb ===
 - commit 7ee8d8ca4ec082736d1bb70425ced58dbd553abb
 - Author: cdcordobaa <cdcordobaa@unal.edu.co>
 - ... +15 lines (ctrl+o to expand)
- Searched for 1 pattern (ctrl+o to expand)
- No existing **CHANGELOG.md** found. Writing a new one now.
- **Write**(CHANGELOG.md)
 - └ Wrote 33 lines to **CHANGELOG.md**
 - 1 # Changelog
 - 2
 - 3 All notable changes to this project will be documented in this file.
 - 4
 - 5 ---
 - 6
 - 7 ## [Unreleased] - 2026-02-25
 - 8
 - 9 > Base commit: `2e8fbff` - no tags found in repository.

Lo que NO delegar al agente

Por más potente que sea el setup — hay decisiones que quedan en el humano

Merge a main / master

El agente prepara el PR, no lo fusiona.

El agente puede:

✓ Crear el PR completo con descripción

Ejecutar en producción

El agente genera el comando, el humano lo ejecuta.

El agente puede:

✓ Generar y explicar el comando exacto

Modificar configuración de seguridad

IAM, permisos, secrets — siempre revisión humana primero.

El agente puede:

✓ Listar cambios necesarios para revisión

Operaciones destructivas irreversibles

Borrar datos, truncar tablas, eliminar recursos cloud.

El agente puede:

✓ Preparar el script con dry-run para validar

Recursos y próximos pasos

Documentación oficial

MCP Spec

`modelcontextprotocol.io`

Servers community

`github.com/modelcontextprotocol/servers`

Claude Code docs

`docs.anthropic.com/claude-code`

Servers útiles para empezar

`@mcp/server-filesystem`

— Archivos locales

`@mcp/server-github`

— GitHub API completa

`@mcp/server-postgres`

— PostgreSQL con
introspección

`@mcp/server-brave-search`

— Búsqueda web

`@mcp/mcp-server-sqlite`

— SQLite para
prototipos

Recomendaciones para Experimentar

→ Construir MCP servers custom en TypeScript y Python → MCP + OAuth para autenticación de servicios externos → Pipelines CI/CD donde el agente es participante activo → Skills de equipo: code review, generación de tests, release workflows



Ahora a construir.

MCPs + Skills + Claude Code

HARDCORE AI BY 30X